GUIA DE IMPLEMENTACAO

Notificacoes e vibracao no Encomendas

Referencia para adaptar o fluxo do SenhaHub a um site instalavel no celular (PWA). Inclui as dependencias, configuracao VAPID e codigos-base de navegador e servidor.

Premissa: este guia considera que o Encomendas e uma aplicacao web/PWA. Para um app nativo em React Native ou Flutter, o mecanismo e diferente.

O que instalar

Para a implementacao Web Push, a unica dependencia externa nova e **web-push**, instalada no backend Node.js. As APIs usadas no navegador ja fazem parte do browser.

Componente	Para que serve	Instalar?
web-push	Envia notificacoes do servidor para as inscricoes dos navegadores usando Web Push e VAPID.	Sim: npm install web-push
PushManager / Push API	Cria a inscricao do dispositivo para receber push.	Nao; API do navegador
Notification API	Pede permissao e exibe notificacoes.	Nao; API do navegador
Service Worker	Recebe o evento push mesmo sem a pagina aberta e mostra a notificacao.	Nao; API do navegador
Vibration API	Solicita vibracao do aparelho via navigator.vibrate().	Nao; suporte varia

Como as partes se conectam

1	A pessoa toca em Ativar notificacoes. O navegador pede permissao e cria uma inscricao Push.
2	O frontend envia a inscricao autenticada para o backend, que a salva associada a conta no banco existente.
3	Quando o pedido muda, o servidor usa web-push e as chaves VAPID para entregar o evento.
4	O Service Worker mostra a notificacao. O navegador pode vibrar conforme suporte e configuracao do aparelho.

Push requer HTTPS em producao e um Service Worker ativo. Em desenvolvimento, localhost costuma ser aceito como contexto seguro. A vibracao deve ser tratada como opcional.

1. Dependencia e chaves VAPID

Instale a biblioteca no projeto Node que envia as notificacoes. Em seguida, gere as chaves VAPID uma unica vez e guarde a chave privada apenas no ambiente do servidor.

Terminal

```
npm install web-push
npx web-push generate-vapid-keys --json
```

Cadastre os valores gerados no ambiente do servidor. Use um e-mail real de contato no subject.

.env do servidor (exemplo)

```
VAPID_PUBLIC_KEY=chave_publica_gerada
VAPID_PRIVATE_KEY=chave_privada_gerada
VAPID_SUBJECT=mailto:suporte@seudominio.com
```

> A chave publica pode ser enviada ao frontend. Nunca exponha VAPID_PRIVATE_KEY em variaveis publicas, JavaScript do navegador, repositorio ou resposta de API.

2. Frontend: pedir permissao e inscrever o aparelho

O pedido de permissao deve acontecer depois de um toque da pessoa. A constante VAPID_PUBLIC_KEY abaixo deve ser preenchida pelo mecanismo publico de configuracao que o framework do Encomendas ja utiliza.

HTML

```
<button id="enable-push">Ativar notificacoes</button>
```

JavaScript do frontend

```
const VAPID_PUBLIC_KEY = "CHAVE_PUBLICA_VAPID";

function urlBase64ToUint8Array(value) {
  const padding = "=".repeat((4 - value.length % 4) % 4);
  const base64 = (value + padding).replace(/-/g, "+").replace(/_/g, "/");
  const raw = window.atob(base64);
  return Uint8Array.from(raw, (character) => character.charCodeAt(0));
}

document.querySelector("#enable-push").addEventListener("click", async () => {
  if (!("serviceWorker" in navigator) || !("PushManager" in window)) {
    alert("Este navegador nao oferece suporte a notificacoes push.");
    return;
  }

  const permission = await Notification.requestPermission();
  if (permission !== "granted") return;

  const registration = await navigator.serviceWorker.register("/sw.js");
  const subscription =
    (await registration.pushManager.getSubscription()) ||
    (await registration.pushManager.subscribe({
      userVisibleOnly: true,
      applicationServerKey: urlBase64ToUint8Array(VAPID_PUBLIC_KEY),
    }));

  await fetch("/api/push/subscribe", {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify(subscription),
    credentials: "same-origin",
  });
});
```

O endpoint /api/push/subscribe deve exigir sessao autenticada e associar a inscricao ao usuario da sessao, sem confiar em um userId enviado no corpo da requisicao.

3. Service Worker: receber e exibir o push

Coloque o arquivo no caminho publico /sw.js (ou equivalente no framework). O backend envia um payload JSON; o Service Worker mostra a notificacao e abre o pedido quando a pessoa toca nela.

/public/sw.js

```
self.addEventListener("push", (event) => {
  let data = {};
  try {
    data = event.data?.json() || {};
  } catch {
    data = { title: "Encomendas", body: event.data?.text() || "" };
  }

  const options = {
    body: data.body || "Ha uma atualizacao no seu pedido.",
    icon: "/icons/icon-192.png",
    tag: data.id || "encomendas",
    data: { url: data.url || "/" },
  };

  if (data.urgent) options.vibrate = [180, 90, 180];

  event.waitUntil(
    self.registration.showNotification(data.title || "Encomendas", options)
  );
});

self.addEventListener("notificationclick", (event) => {
  event.notification.close();
  const target = new URL(
    event.notification.data?.url || "/",
    self.location.origin
  );
  if (target.origin !== self.location.origin) {
    target.href = `${self.location.origin}/`;
  }
  event.waitUntil(self.clients.openWindow(target.href));
});
```

4. Vibração enquanto a página está aberta

Para chamadas imediatas quando a pessoa já está usando o site, chame esta funcao ao detectar a mudanca do pedido:

JavaScript do frontend

```
function vibrateOrderAlert() {
  if (typeof navigator.vibrate === "function") {
    navigator.vibrate([180, 90, 180]);
  }
}

// Exemplo: chamar quando o pedido ficar pronto.
vibrateOrderAlert();
```

> O suporte a navigator.vibrate e limitado em alguns navegadores. A opcao vibrate da notificacao tambem pode ser ignorada pelo sistema ou pelas configuracoes do aparelho. Mantenha titulo, texto e som visual do alerta como sinais principais.

5. Backend: configurar e enviar

Este exemplo e um modulo Node.js CommonJS. Conecte-o ao evento real de mudanca de status do pedido e use as inscricoes salvas para o usuario destinatario.

server/push-service.cjs

```
const webPush = require("web-push");

webPush.setVapidDetails(
  process.env.VAPID_SUBJECT,
  process.env.VAPID_PUBLIC_KEY,
  process.env.VAPID_PRIVATE_KEY
);

async function sendOrderPush(subscription, order) {
  const payload = {
    id: `order-${order.id}-${order.status}`,
    title: "Atualizacao do pedido",
    body: `O pedido ${order.number} esta ${order.statusLabel}.`,
    url: `/pedidos/${order.id}`,
    urgent: order.status === "pronto",
  };

  return webPush.sendNotification(
    subscription,
    JSON.stringify(payload),
    { TTL: 60 }
  );
}

module.exports = { sendOrderPush };
```

Associe a chamada ao fluxo de negocio do Encomendas. Exemplo: ao marcar um pedido como pronto, carregue as inscricoes push daquela conta e chame sendOrderPush para cada uma.

Persistencia de inscricoes (PostgreSQL, exemplo)

Use o banco que o Encomendas ja possui. Se for PostgreSQL, uma tabela simples pode guardar o usuario, o endpoint e o JSON recebido do navegador. Ajuste user_id para o tipo de identificador da tabela de usuarios.

SQL

```
CREATE TABLE push_subscriptions (
  id bigserial PRIMARY KEY,
  user_id text NOT NULL,
  endpoint text NOT NULL UNIQUE,
  subscription jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

-- O endpoint POST /api/push/subscribe deve fazer upsert:
INSERT INTO push_subscriptions (user_id, endpoint, subscription)
VALUES ($1, $2, $3::jsonb)
ON CONFLICT (endpoint) DO UPDATE SET
  user_id = EXCLUDED.user_id,
  subscription = EXCLUDED.subscription;
```

Rotas e protecoes necessarias

POST /api/push/subscribe: autenticar a sessao, validar endpoint e keys.p256dh/keys.auth, salvar a inscricao do usuario atual e retornar sucesso. **Evento do pedido:** enviar a notificacao no servidor, nao diretamente do navegador. **Falhas:** se web-push retornar HTTP 404 ou 410, remover a inscricao vencida; registrar outras falhas para diagnostico.

6. Checklist de integracao

- Instalar web-push somente no backend Node.js.
- Gerar VAPID uma vez; guardar a chave privada como segredo do servidor.
- Servir o site por HTTPS em producao e publicar o Service Worker na raiz ou no escopo correto.
- Pedir permissao somente depois da acao da pessoa.
- Salvar a inscricao no banco associada a conta autenticada.
- Enviar o push do backend quando o evento do pedido acontecer.
- Remover inscricoes expiradas (HTTP 404/410) e oferecer opcao para desativar alertas.
- Testar em aparelhos e navegadores reais; push e vibracao variam por plataforma.

> Este material e um ponto de partida. A rota, autenticacao, persistencia e configuracao publica da chave devem seguir a arquitetura que ja existe no Encomendas.

Referencias tecnicas

web-push: instalacao, VAPID e envio
https://github.com/web-push-libs/web-push/blob/master/README.md

MDN: Push API
https://developer.mozilla.org/en-US/docs/Web/API/Push_API

MDN: PushManager
https://developer.mozilla.org/en-US/docs/Web/API/PushManager

MDN: showNotification()
https://developer.mozilla.org/en-US/docs/Web/API/ServiceWorkerRegistration/showNotification

MDN: Vibration API
https://developer.mozilla.org/en-US/docs/Web/API/Vibration_API

Guia preparado a partir do fluxo de notificacoes e vibracao encontrado no SenhaHub, com exemplos portaveis para integracao no Encomendas.